Container Isolation: How Docker Tricks the OS

Case Study: Docker Containers & Cloud Computing

Operating Systems Class Presentation

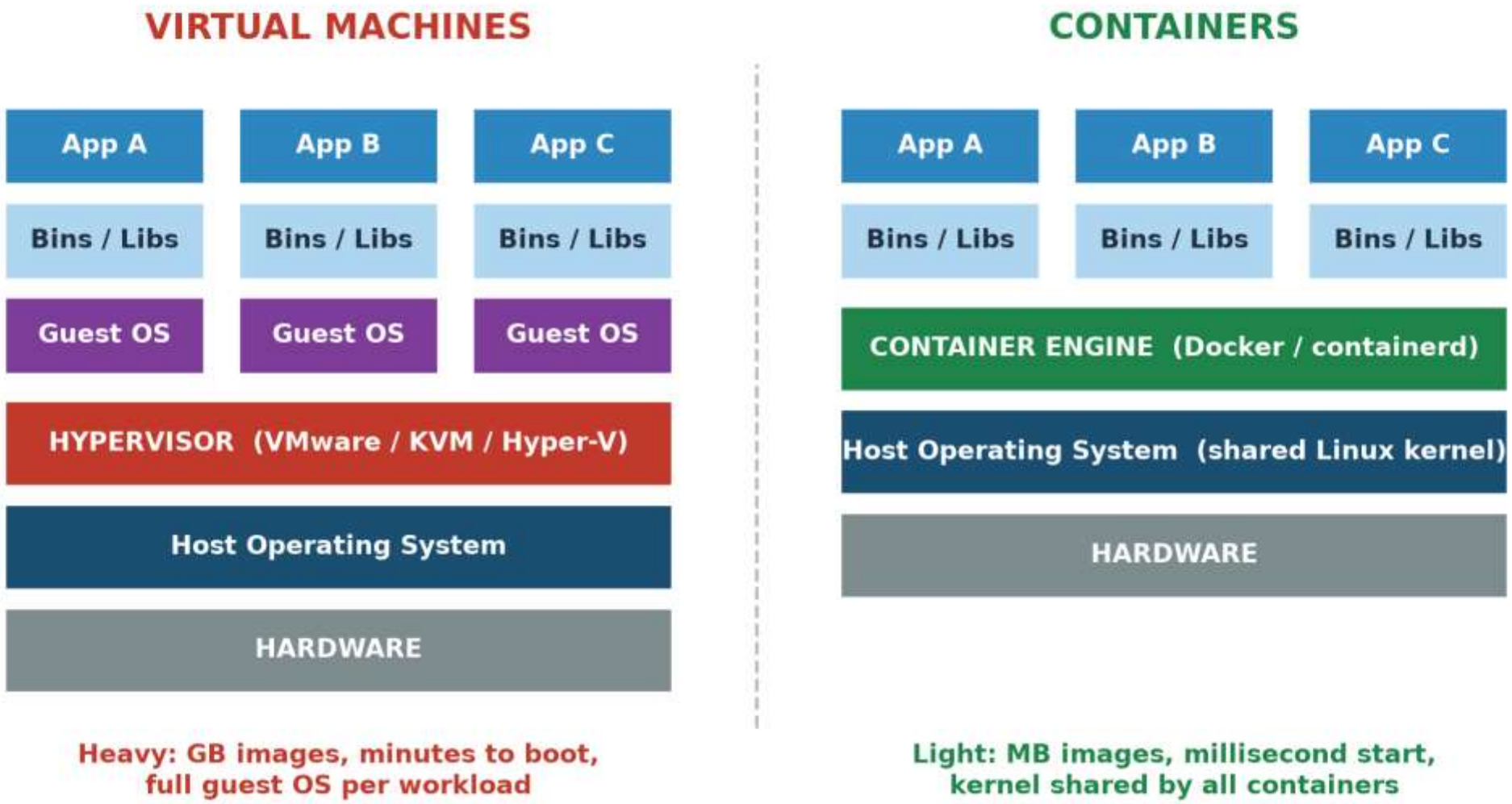
Agenda

- Introduction to containers
- Virtual machines vs containers
- Linux namespaces
- Control groups (cgroups)
- Overlay filesystems
- Security layers
- Docker execution model
- Containers in cloud computing
- Limitations and security concerns
- Conclusion

What Is a Container?

- A lightweight, isolated user-space environment
- Shares the host operating system kernel
- Packages application code, libraries, and dependencies
- Faster and lighter than a full virtual machine
- Uses Linux kernel features such as namespaces and cgroups

Virtual Machines vs Containers



Virtual Machines vs Containers

- Virtual machines emulate complete hardware
  - Require a hypervisor such as VMware, KVM, or VirtualBox
  - Each VM runs a full guest operating system
- Containers use OS-level isolation
  - Containers share the host kernel
  - Containers start in milliseconds to seconds
- VMs provide stronger isolation but higher overhead
- Containers provide faster deployment and higher density

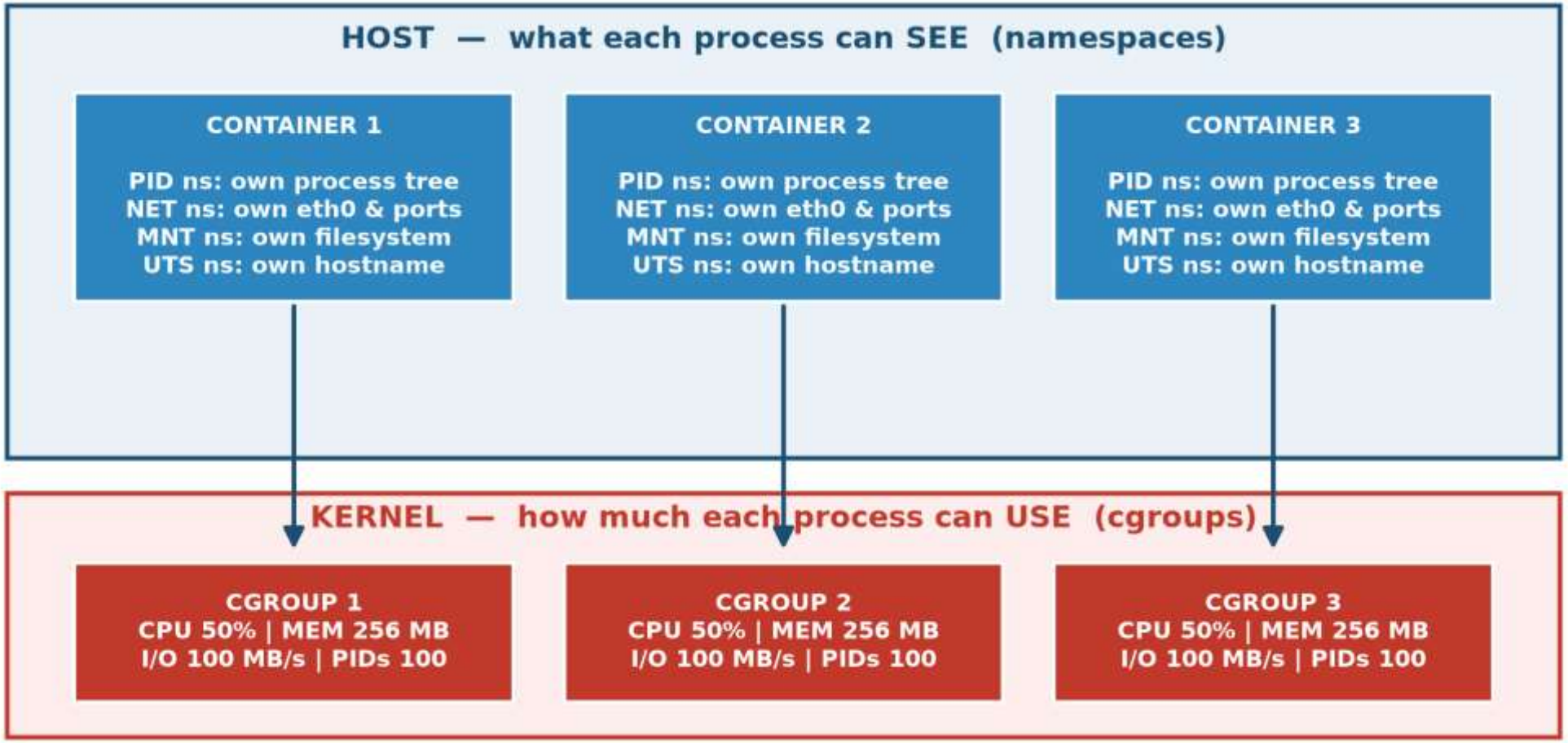
Pillar 1: Linux Namespaces

- Namespaces control what a process can see
- PID namespace: isolates process IDs
- NET namespace: isolates network interfaces and ports
- MNT namespace: isolates mount points
- UTS namespace: isolates hostname
- IPC namespace: isolates inter-process communication
- USER namespace: isolates user and group IDs
- Each container gets a private view of the system

Pillar 2: Control Groups (cgroups)

- Cgroups control how much resource a container can use
- CPU: limit percentage of processor time
- Memory: set hard memory limits
- Block I/O: limit disk read/write bandwidth
- PIDs: limit number of processes
- Devices: allow or deny device access
- Prevents one container from starving others

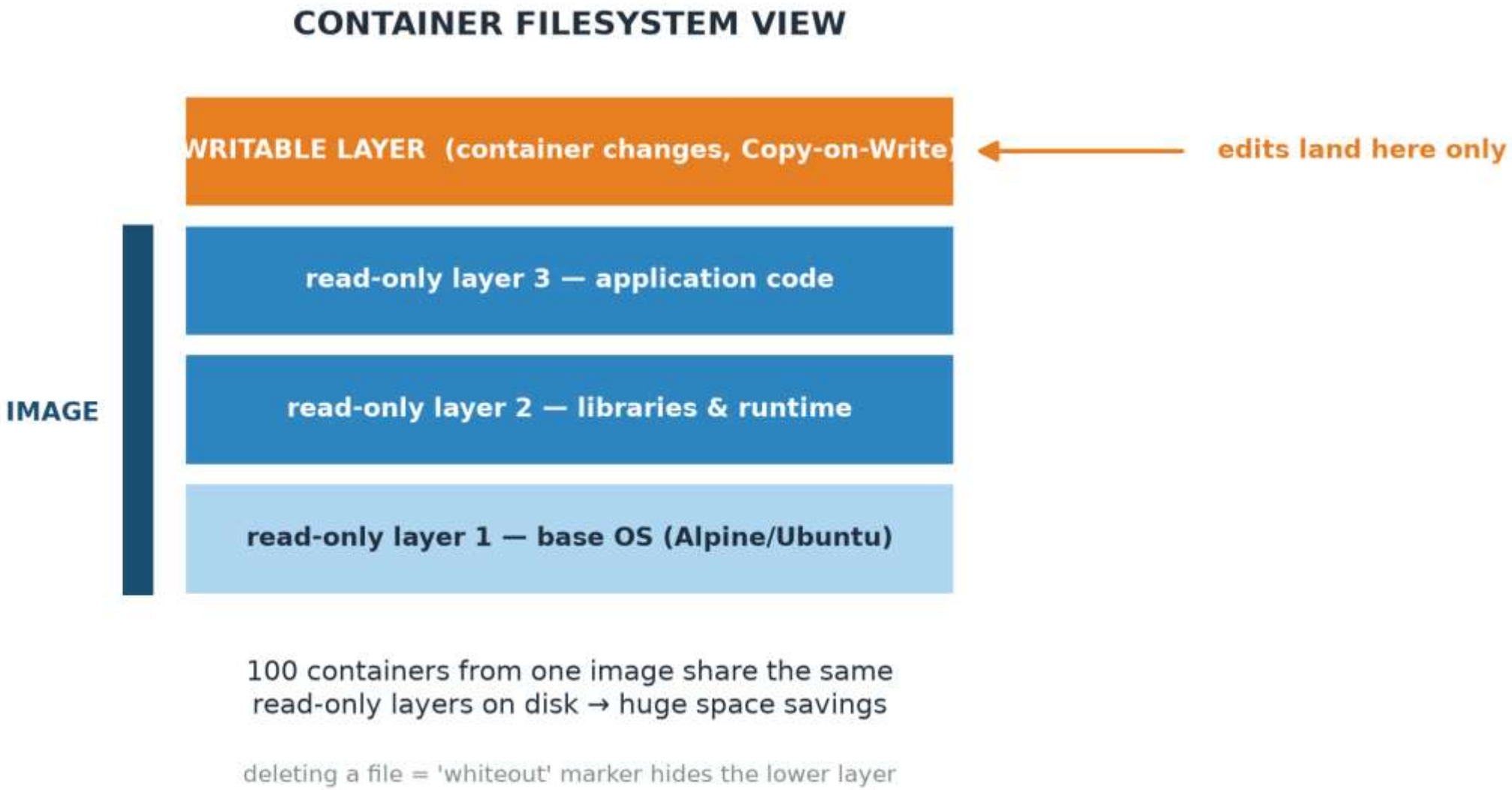
Namespaces vs Cgroups: See vs Use



Pillar 3: Overlay Filesystems

```
Container Writable Layer
---------------------------------
Read-only Layer 3: Application code
Read-only Layer 2: Libraries/runtime
Read-only Layer 1: Base OS image
```

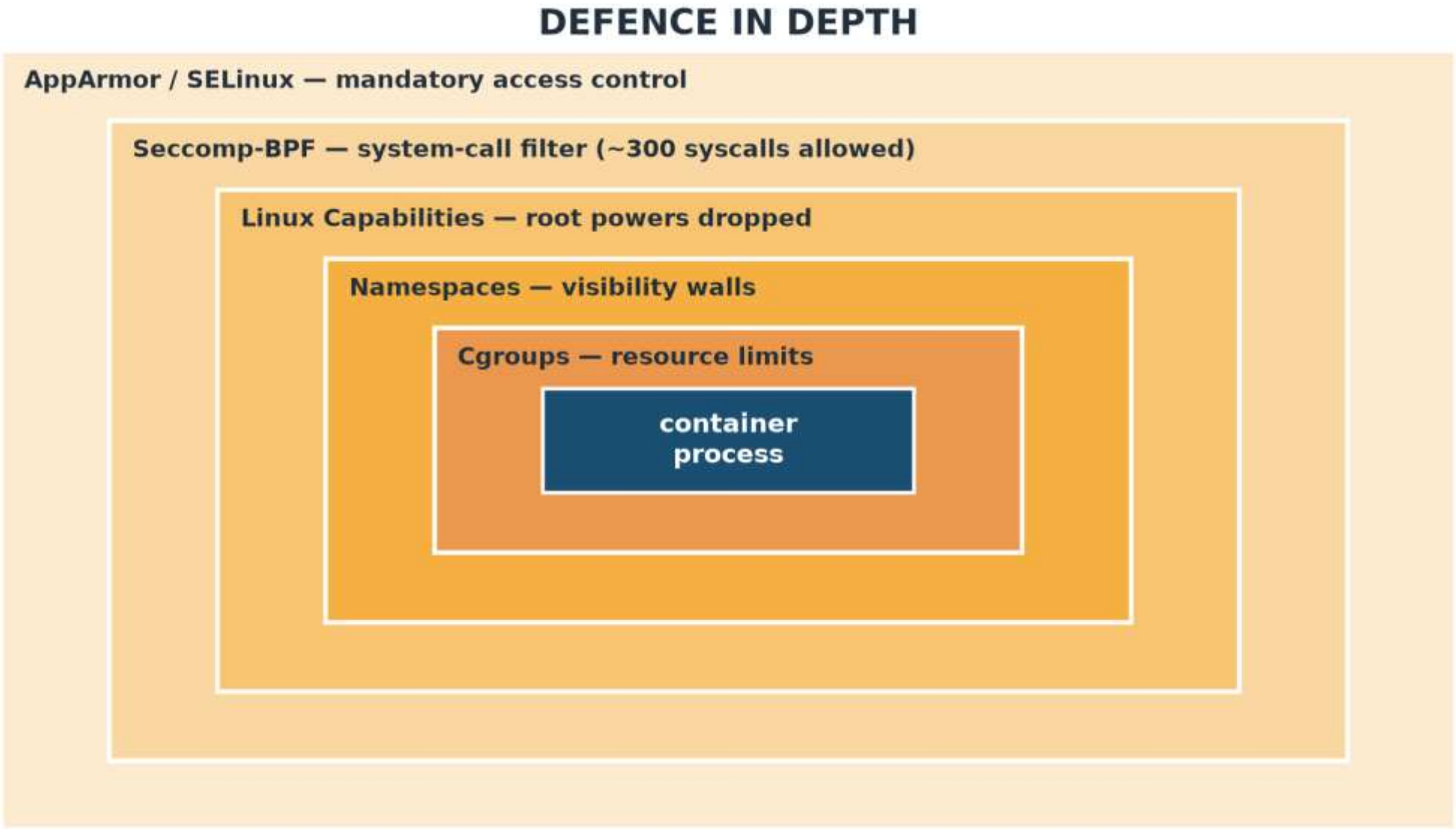
Overlay Filesystem Layers



Pillar 4: Security Layers

- Cgroups limit resource usage
- Namespaces isolate visibility
- Linux capabilities remove unnecessary root privileges
- Seccomp-BPF restricts allowed system calls
- AppArmor or SELinux enforces mandatory access control
- Docker uses defense-in-depth

Defence in Depth

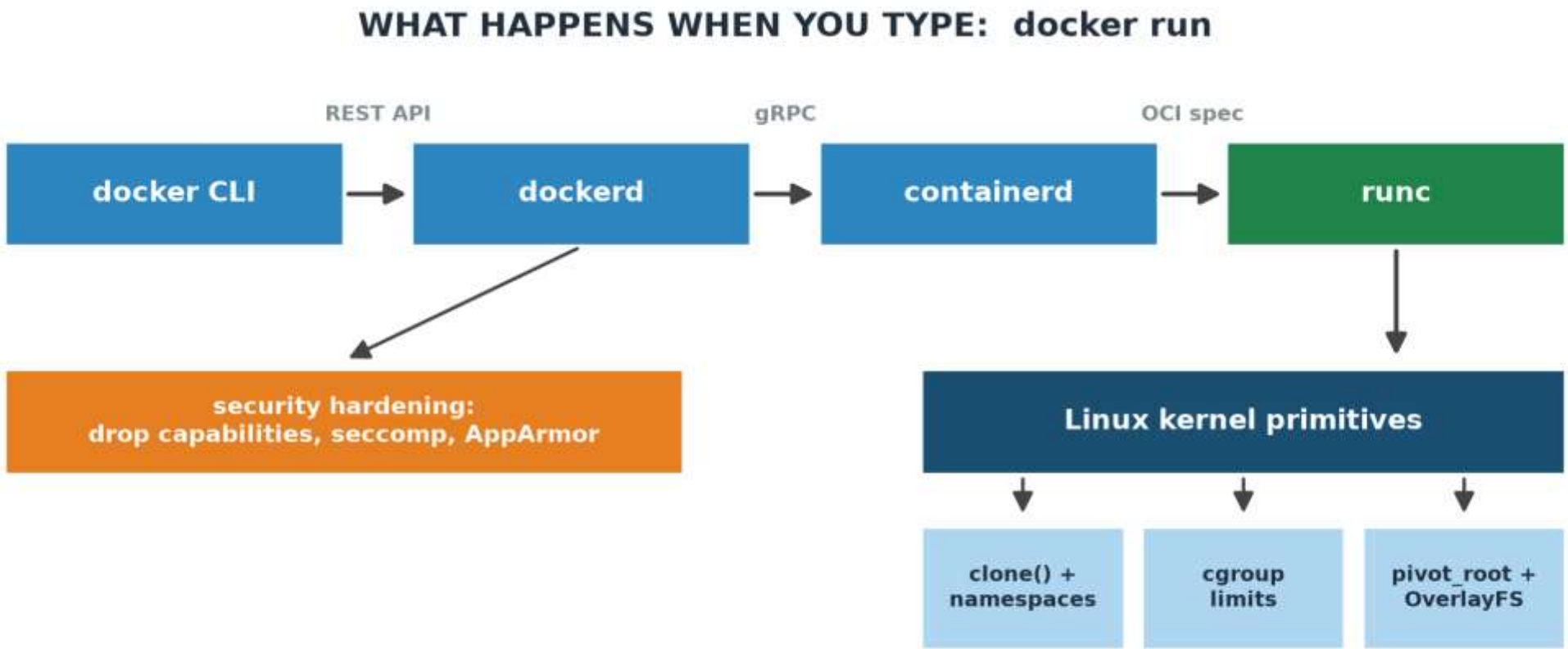


How Docker Launches a Container

```
docker CLI -> dockerd -> containerd -> runc

runc performs:
  1. clone() with namespace flags
  2. cgroup resource setup
  3. filesystem setup with pivot_root()
  4. capability dropping and seccomp filtering
  5. exec() of container entrypoint
```

Inside docker run



runc creates the namespaces, sets cgroups, pivots root and execs the app

Example Docker Command

```
docker run -d --name web \
  --memory=256m \
  --cpus=0.5 \
  --read-only \
  --tmpfs /tmp \
  --security-opt=no-new-privileges \
  nginx:alpine
```

Class Demo: Observing Isolation

```
docker run -d --name demo --memory=128m --cpus=0.5 nginx:alpine

docker top demo

ps aux | grep nginx

docker inspect demo --format '{{.HostConfig.Memory}}'

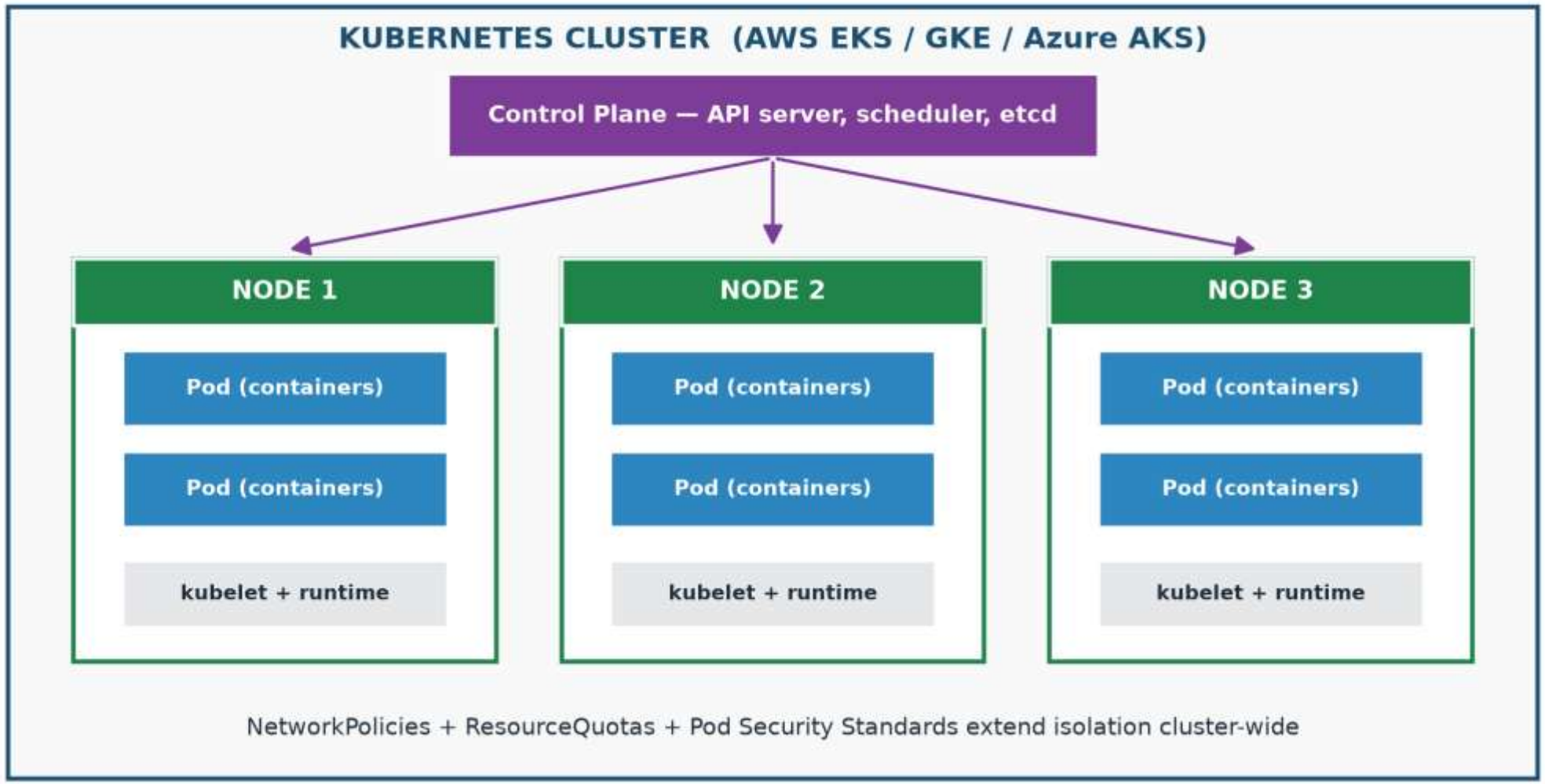
PID=$(docker inspect -f '{{.State.Pid}}' demo)

ls -l /proc/$PID/ns/
```

Containers in Cloud Computing

- Cloud providers use containers for fast scaling
- Many workloads can share the same physical server
- Kubernetes manages containers across many machines
- Managed services include AWS EKS, Google GKE, Azure AKS
- Serverless platforms often use containers or microVMs
- Isolation enables multi-tenant cloud environments

Kubernetes: Isolation at Cloud Scale

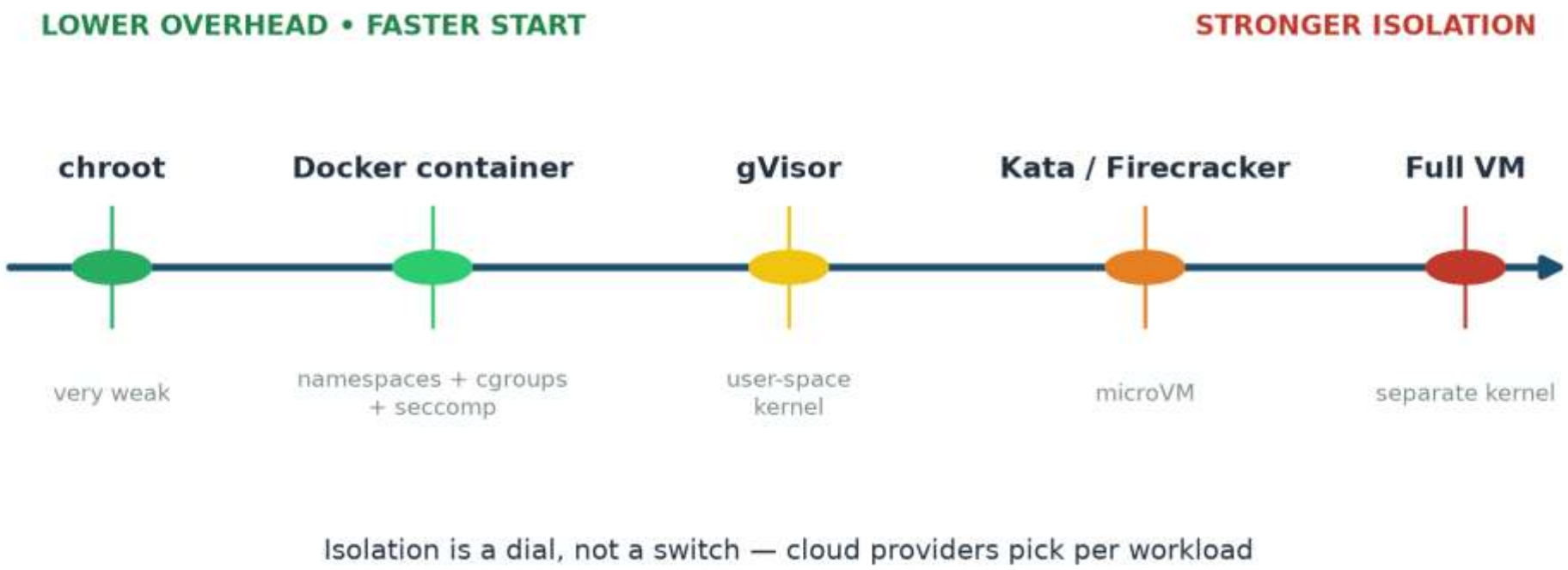


Isolation Strength Spectrum

```
Weaker Isolation <----------------------------------------> Stronger Isolation

chroot        Docker        gVisor/user-space kernel        Kata/Firecracker        Full VM
```

Isolation Strength Spectrum



Isolation is a dial, not a switch — cloud providers pick per workload

Limitations and Security Concerns

- Containers share the host kernel
- A kernel vulnerability may affect all containers
- Privileged containers weaken isolation
- Side-channel attacks may cross container boundaries
- Malicious images can introduce vulnerabilities
- Network traffic may need encryption

Key Takeaways

- Containers are isolated processes, not full virtual machines
- Namespaces control what a container can see
- Cgroups control how much resource a container can use
- Overlay filesystems provide private filesystem views
- Security layers reduce privileges and system-call access
- Containers are central to modern cloud computing

Questions?

- Thank you for listening
- Any questions?